

Queuemaxxing Lite — Technical Assignment

Overview

Build a simple HTTP-based queue service using **Python** and **FastAPI**. The service must support creating queues, pushing items, claiming items in a configurable order, and deleting items.

The assignment prioritizes **correctness, persistence, and concurrency safety** over throughput or distributed-systems features.

Scope

In scope

- FIFO or LIFO ordering
- Priority-based ordering
- Delayed message availability
- Persistent storage across application restarts
- Safe handling of concurrent HTTP requests
- Local filesystem-based storage
- No external database or queue system

Out of scope

- Authentication / authorization
 - Distributed deployment or replication
 - High availability
 - Consumer groups or Pub/Sub
 - Long polling
 - Retry, ack/nack, or visibility timeout
 - Web UI
 - Docker/Kubernetes (optional bonus only)
-

Technology Requirements

Required

- Python 3.11 +
- FastAPI
- Uvicorn
- Pydantic
- Python standard library

Allowed

- pytest
- httpx

Not allowed

- Redis, PostgreSQL, MySQL, MongoDB, SQLite
 - RabbitMQ, Kafka, Pulsar, Celery, Amazon SQS
 - Any external database or queue system
-

HTTP API

The service must expose exactly four endpoints:

Method	Path	Purpose
POST	/create_queue	Create a new queue
POST	/queues/{id}/push	Add an item to a queue
GET	/queues/{id}/claim	Atomically retrieve and remove the next eligible item
DELETE	/queues/{id}/ {queue_item_id}	Remove a specific item from a queue

Use 201 Created for resource creation and 204 No Content for successful operations with no response body.

1. Create Queue

POST /create_queue
Content-Type: application/json

Request body

```
{
  "name": "email-jobs",
  "ordering": "fifo",
  "priority_enabled": true
}
```

Field	Type	Required	Rules
name	string	Yes	1–128 characters
ordering	string	Yes	Must be "fifo" or "lifo"
priority_enabled	boolean	No	Defaults to false

Response — 201 Created

```
{
  "id": "0f9e1d95-46f3-4857-a5a0-c3b437d73f1b",
  "name": "email-jobs",
  "ordering": "fifo",
  "priority_enabled": true,
}
```

```
  "created_at": "2026-08-21T01:16:00Z"
}
```

If the queue name already exists, return 409 Conflict.

2. Push Item

POST /queues/{id}/push
Content-Type: application/json

Request body

```
{
  "payload": {
    "task": "send_email",
    "recipient": "user@example.com"
  },
  "priority": 10,
  "delay_seconds": 5
}
```

Field	Type	Required	Rules
payload	JSON object	Yes	Arbitrary job data
priority	integer	No	Defaults to 0
delay_seconds	integer	No	Defaults to 0, must be ≥ 0

Response — 201 Created

```
{
  "id": "9bc6704d-d452-4ec2-8c65-5146b12f0095",
  "queue_id": "0f9e1d95-46f3-4857-a5a0-c3b437d73f1b",
  "priority": 10,
  "created_at": "2026-08-21T01:16:00Z",
  "available_at": "2026-08-21T01:16:05Z",
  "status": "delayed"
}
```

If delay_seconds is 0, status should be "ready".

If the queue does not exist, return 404 Not Found.

3. Claim Item

GET /queues/{id}/claim

Retrieves one eligible item and removes it from the queue atomically.

Response — 200 OK (item available)

```
{
  "id": "9bc6704d-d452-4ec2-8c65-5146b12f0095",
```

```
"queue_id": "0f9e1d95-46f3-4857-a5a0-c3b437d73f1b",
"payload": {
  "task": "send_email",
  "recipient": "user@example.com"
},
"priority": 10,
"sequence": 1,
"created_at": "2026-08-21T01:16:00Z",
"available_at": "2026-08-21T01:16:05Z"
}
```

Response — 204 No Content (nothing available)

No response body.

A delayed item whose `available_at` has not passed must never be claimed, regardless of priority.

Note: `GET /claim` mutates state by removing an item. This deviates from strict REST semantics but is required by the API contract.

4. Delete Item

```
DELETE /queues/{id}/{queue_item_id}
```

Removes a specific item, whether it is ready or still delayed.

Response — 204 No Content on success.

Scenario	Response
Queue does not exist	404 Not Found
Item does not exist	404 Not Found
Item belongs to a different queue	404 Not Found

Queue Ordering Rules

Each queue maintains a monotonically increasing `sequence` counter, assigned when an item is pushed.

Claims only consider items where:

```
available_at <= current_time
```

FIFO (no priority): lowest sequence claimed first.

LIFO (no priority): highest sequence claimed first.

Priority FIFO: sort by priority descending, then sequence ascending.

Priority LIFO: sort by priority descending, then sequence descending.

Example

Item Priority Sequence

A	1	1
B	10	2
C	10	3
D	5	4

- Priority FIFO claim order: B, C, D, A
 - Priority LIFO claim order: C, B, D, A
-

Persistence Requirements

All queues and items must survive an application restart.

Verification scenario:

1. Create a queue.
2. Push an item.
3. Stop the application.
4. Restart the application.
5. Call `/claim`.
6. The previously pushed item must still be retrievable.

Storage Constraint

No database or external queue is allowed. All state must be persisted to the local filesystem, managed directly by the application.

Suggested project structure

```
project/
├── app/
│   ├── main.py
│   ├── models.py
│   ├── queue_service.py
│   └── storage.py
├── data/
│   └── queue_state.json
├── tests/
└── README.md
```

Recommended Persistence Strategy

Store all state in a single JSON file:

```
{
  "queues": {
    "queue-id": {
      "id": "queue-id",
      "name": "email-jobs",
      "ordering": "fifo",
```

```

        "priority_enabled": true,
        "sequence_counter": 2
    }
},
"items": {
    "item-id": {
        "id": "item-id",
        "queue_id": "queue-id",
        "payload": { "task": "send_email" },
        "priority": 10,
        "sequence": 1,
        "created_at": "2026-08-21T01:16:00Z",
        "available_at": "2026-08-21T01:16:05Z"
    }
}
}

```

Write sequence for every mutation:

1. Acquire lock
2. Update in-memory state
3. Serialize state to a temporary file
4. Flush and fsync the file
5. Atomically replace the state file (`os.replace`)
6. Release lock
7. Return HTTP response

Writing to a temporary file followed by an atomic rename reduces the risk of a corrupted state file if the application crashes mid-write.

Concurrency Requirements

The service must remain correct under concurrent HTTP requests.

At minimum, use an `asyncio.Lock` to guard state mutations:

```
state_lock = asyncio.Lock()
```

All endpoints that mutate state must acquire the lock during the critical section:

- POST `/create_queue`
- POST `/queues/{id}/push`
- GET `/queues/{id}/claim`
- DELETE `/queues/{id}/{queue_item_id}`

Claim must be atomic. Two concurrent requests must never claim the same item.

Required claim sequence:

```

Acquire lock
-> find eligible items
-> select the correct item using the ordering policy

```

- > remove the selected item from state
- > persist the updated state
- > release lock
- > return the claimed item

Delivery Semantics

Because the API has no `ack`, `nack`, or visibility timeout, this queue provides **at-most-once delivery**:

- An item is deleted as soon as `/claim` returns it successfully.
- If a consumer crashes after receiving the claim response, the item is lost.
- Automatic replay is not supported.
- This is an accepted trade-off to keep the assignment scope small.

A future extension with `ack/nack`, lease IDs, and visibility timeouts would upgrade this to **at-least-once delivery**.

Required Tests

Queue creation

- Successfully creates a queue.
- Duplicate queue name returns 409 Conflict.
- Invalid `ordering` value returns a validation error.

FIFO and LIFO

- Push A, B, C to a FIFO queue; claims return A, B, C in order.
- Push A, B, C to a LIFO queue; claims return C, B, A in order.

Priority

- Push items with priority 1, 10, and 5.
- With priority enabled, claim order is 10, 5, 1.
- With priority disabled, priority has no effect on order.

Delay

- Push an item with `delay_seconds = 5`.
- An immediate claim returns 204 No Content.
- After the delay elapses, the claim returns the item.
- Tests may mock the clock instead of using real sleeps.

Persistence

- Push an item.
- Reload service state from the persisted file.
- The item remains claimable.

Concurrency

- Push at least 100 items.
 - Issue concurrent claim requests from multiple clients.
 - Each item is returned at most once.
 - The total number of successful claims equals the number of pushed items.
-

Deliverables

The GitHub repository must include:

```
README.md
requirements.txt (or pyproject.toml)
app/
tests/
data/.gitkeep
```

README.md must cover:

- Installation and startup instructions
 - Example `curl` commands for each endpoint
 - Queue ordering semantics
 - Persistence strategy
 - Concurrency strategy
 - Delivery semantics (at-most-once)
 - Known limitations of the implementation
-

Running the Service

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
uvicorn app.main:app --reload
```

FastAPI automatically generates interactive API documentation at:

```
http://127.0.0.1:8000/docs
```